

BABU BANARASI DAS **UNIVERSITY**

2026 - 2027



SCHOOL OF COMPUTER APPLICATIONS

No SQL Project On
Student Management System Using MongoDB

Submitted to:

Mr. Harendra Singh

Department: BCA (DS & AI)

Date: 25th September

Submitted by:

Aman Verma

Roll no: 65

University Roll no: 1250258073

ACKNOWLEDGEMENT

I sincerely express my gratitude to my teacher Mr. Harendra Singh for giving me the opportunity to work on this NoSQL project titled "Student Management System Using MongoDB".

This project helped me understand the basic concepts of NoSQL databases and learn how MongoDB is used to create, retrieve, update, sort and delete data. I also learned how different query operators such as greater than, less than, equal to, AND, and OR are used in MongoDB.

I am thankful to my teacher for the guidance and support provided throughout the completion of this project.

INTRODUCTION

MongoDB is a popular NoSQL database management system that stores data in the form of documents. Unlike traditional relational databases, MongoDB does not require data to be stored in fixed rows and columns.

In this project, a Student Management System has been created using MongoDB. The database contains information about 50 students, including their roll number, name, age, marks, and city.

Different MongoDB operations have been performed on the student data, such as inserting, displaying, updating, deleting, sorting, counting, and searching records. Comparison operators such as \$eq, \$gt, \$lt, \$gte, and \$lte have also been demonstrated.

OBJECTIVES

- To understand the concept of NoSQL databases.
- To learn the basic features of MongoDB.
- To create a database and collection.
- To insert 50 student records.
- To perform CRUD operations.
- To use comparison operators.
- To perform sorting and filtering.
- To understand AND and OR operations.

1. CREATE DATABASE

MongoDB Query:

use Students

Explanation: Creates or switches to the Students database.

```
MongoDB Shell — Students

> MONGOSH
use Students
switched to db Students
```

2. CREATE COLLECTION

MongoDB Query:

db.createCollection("students")

Explanation: Creates a collection named students.

```
MongoDB Shell — Students

> MONGOSH
db.createCollection("students")
{ ok: 1 }
```

3. INSERT 50 STUDENTS

MongoDB Query:

db.students.insertMany([...])

Explanation: Inserts 50 student documents into the collection.

```
MongoDB Shell — Students

> MONGOSH
db.students.insertMany([
  { _id: ObjectId('6f1a2e9c78f0f98eb52df200'), roll: 1, name: 'Ritika', age: 18, marks: 51, city:
'Ayodhya' },
  { _id: ObjectId('6f1a2e9c78f0f98eb52df201'), roll: 2, name: 'Devansh', age: 19, marks: 64, city:
'Varanasi' },
  { _id: ObjectId('6f1a2e9c78f0f98eb52df202'), roll: 3, name: 'Tanya', age: 18, marks: 93, city:
'Gonda' },
  // ... 47 more student documents
])
{ acknowledged: true, insertedCount: 50 }
```

4. DISPLAY ALL STUDENTS

MongoDB Query:

```
db.students.find()
```

Explanation: Retrieves all documents from the students collection.

```
MongoDB Shell — Students

> MONGOSH
db.students.find()
{ _id: ObjectId('6f1a2e9c78f0f98eb52df200'), roll: 1, name: 'Ritika', age: 18, marks: 51, city: 'Ayodhya' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df201'), roll: 2, name: 'Devansh', age: 19, marks: 64, city: 'Varanasi' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df202'), roll: 3, name: 'Tanya', age: 18, marks: 93, city: 'Gonda' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df203'), roll: 4, name: 'Yash', age: 18, marks: 87, city: 'Jaunpur' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df204'), roll: 5, name: 'Kritika', age: 18, marks: 60, city: 'Kanpur' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df205'), roll: 6, name: 'Aarav', age: 19, marks: 64, city: 'Gonda' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df206'), roll: 7, name: 'Simran', age: 18, marks: 85, city: 'Prayagraj' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df207'), roll: 8, name: 'Shivam', age: 21, marks: 88, city: 'Bareilly' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df208'), roll: 9, name: 'Ishita', age: 20, marks: 88, city: 'Varanasi' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df209'), roll: 10, name: 'Rohit', age: 21, marks: 55, city: 'Ayodhya' }
Type "it" for more (40 more documents)
```

5. EQUAL TO --- \$eq

MongoDB Query:

```
db.students.find({ marks: { $eq: 88 } })
```

Explanation: Finds students whose marks are exactly 88.

```
MongoDB Shell — Students

> MONGOSH
db.students.find({ marks: { $eq: 88 } })
{ _id: ObjectId('6f1a2e9c78f0f98eb52df207'), roll: 8, name: 'Shivam', age: 21, marks: 88, city: 'Bareilly' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df208'), roll: 9, name: 'Ishita', age: 20, marks: 88, city: 'Varanasi' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df217'), roll: 24, name: 'Karan', age: 18, marks: 88, city: 'Varanasi' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df227'), roll: 40, name: 'Gaurav', age: 21, marks: 88, city: 'Kanpur' }
```

6. GREATER THAN --- \$gt

MongoDB Query:

```
db.students.find({ marks: { $gt: 85 } })
```

Explanation: Finds students whose marks are greater than 85.

```
MongoDB Shell — Students

> MONGOSH
db.students.find({ marks: { $gt: 85 } })
{ _id: ObjectId('6f1a2e9c78f0f98eb52df202'), roll: 3, name: 'Tanya', age: 18, marks: 93, city: 'Gonda' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df203'), roll: 4, name: 'Yash', age: 18, marks: 87, city: 'Jaunpur' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df207'), roll: 8, name: 'Shivam', age: 21, marks: 88, city: 'Bareilly' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df208'), roll: 9, name: 'Ishita', age: 20, marks: 88, city: 'Varanasi' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df20f'), roll: 16, name: 'Rajat', age: 20, marks: 90, city: 'Agra' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df210'), roll: 17, name: 'Swati', age: 20, marks: 86, city: 'Prayagraj' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df214'), roll: 21, name: 'Niharika', age: 21, marks: 90, city: 'Gorakhpur' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df216'), roll: 23, name: 'Radhika', age: 19, marks: 92, city: 'Ayodhya' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df217'), roll: 24, name: 'Karan', age: 18, marks: 88, city: 'Varanasi' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df21a'), roll: 27, name: 'Anjali', age: 19, marks: 93, city: 'Gorakhpur' }
(8 more documents)
```

7. LESS THAN --- \$lt

MongoDB Query:

```
db.students.find({ marks: { $lt: 60 } })
```

Explanation: Finds students whose marks are less than 60.

```
MongoDB Shell — Students

> MONGOSH
db.students.find({ marks: { $lt: 60 } })
{ _id: ObjectId('6f1a2e9c78f0f98eb52df200'), roll: 1, name: 'Ritika', age: 18, marks: 51, city: 'Ayodhya' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df209'), roll: 10, name: 'Rohit', age: 21, marks: 55, city: 'Ayodhya' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df20b'), roll: 12, name: 'Aryan', age: 18, marks: 55, city: 'Jaunpur' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df20d'), roll: 14, name: 'Kunal', age: 20, marks: 52, city: 'Bareilly' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df211'), roll: 18, name: 'Manish', age: 18, marks: 52, city: 'Prayagraj' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df212'), roll: 19, name: 'Pooja', age: 20, marks: 55, city: 'Prayagraj' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df225'), roll: 38, name: 'Harshit', age: 21, marks: 55, city: 'Lucknow' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df226'), roll: 39, name: 'Riya', age: 18, marks: 59, city: 'Varanasi' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df230'), roll: 49, name: 'Kabir', age: 19, marks: 59, city: 'Gorakhpur' }
```

8. GREATER THAN OR EQUAL TO --- \$gte

MongoDB Query:

```
db.students.find({ marks: { $gte: 90 } })
```

Explanation: Finds students whose marks are 90 or above.

```
MongoDB Shell — Students

> MONGOSH
db.students.find({ marks: { $gte: 90 } })
{ _id: ObjectId('6f1a2e9c78f0f98eb52df202'), roll: 3, name: 'Tanya', age: 18, marks: 93, city: 'Gonda' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df20f'), roll: 16, name: 'Rajat', age: 20, marks: 90, city: 'Agra' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df214'), roll: 21, name: 'Niharika', age: 21, marks: 90, city: 'Gorakhpur' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df216'), roll: 23, name: 'Radhika', age: 19, marks: 92, city: 'Ayodhya' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df21a'), roll: 27, name: 'Anjali', age: 19, marks: 93, city: 'Gorakhpur' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df21f'), roll: 32, name: 'Nikhil', age: 21, marks: 91, city: 'Bareilly' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df221'), roll: 34, name: 'Arjun', age: 19, marks: 97, city: 'Gonda' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df222'), roll: 35, name: 'Tanvi', age: 20, marks: 97, city: 'Agra' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df22a'), roll: 43, name: 'Rohan', age: 18, marks: 93, city: 'Kanpur' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df22b'), roll: 44, name: 'Aditi', age: 20, marks: 99, city: 'Gorakhpur' }
(1 more documents)
```

9. LESS THAN OR EQUAL TO --- \$lte

MongoDB Query:

```
db.students.find({ marks: { $lte: 70 } })
```

Explanation: Finds students whose marks are 70 or below.


```
MongoDB Shell — Students

> MONGOSH
db.students.find({ marks: { $lte: 70 } })
{ _id: ObjectId('6f1a2e9c78f0f98eb52df200'), roll: 1, name: 'Ritika', age: 18, marks: 51, city: 'Ayodhya' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df201'), roll: 2, name: 'Devansh', age: 19, marks: 64, city: 'Varanasi' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df204'), roll: 5, name: 'Kritika', age: 18, marks: 60, city: 'Kanpur' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df205'), roll: 6, name: 'Aarav', age: 19, marks: 64, city: 'Gonda' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df209'), roll: 10, name: 'Rohit', age: 21, marks: 55, city: 'Ayodhya' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df20a'), roll: 11, name: 'Mahi', age: 19, marks: 63, city: 'Gorakhpur' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df20b'), roll: 12, name: 'Aryan', age: 18, marks: 55, city: 'Jaunpur' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df20d'), roll: 14, name: 'Kunal', age: 20, marks: 52, city: 'Bareilly' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df211'), roll: 18, name: 'Manish', age: 18, marks: 52, city: 'Prayagraj' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df212'), roll: 19, name: 'Pooja', age: 20, marks: 55, city: 'Prayagraj' }
(11 more documents)
```

10. UPDATE OPERATION

MongoDB Query:

```
db.students.updateOne(
  { roll: 5 },
  { $set: { marks: 95 } }
)
```

Explanation: Updates the marks to 95 of the student with roll number 5.

```
MongoDB Shell — Students

> MONGOSH
db.students.updateOne(
  { roll: 5 },
  { $set: { marks: 95 } }
)
{ acknowledged: true, insertedId: null, matchedCount: 1, modifiedCount: 1, upsertedCount: 0 }

// updated document:
{ _id: ObjectId('6f1a2e9c78f0f98eb52df204'), roll: 5, name: 'Kritika', age: 18, marks: 95, city: 'Kanpur' }
```

11. DELETE ONE OPERATION

MongoDB Query:


```
db.students.deleteOne({ roll: 10 })
```

Explanation: Deletes one document matching roll number 10.

```
MongoDB Shell — Students

> MONGOSH
db.students.deleteOne({ roll: 10 })
{ acknowledged: true, deletedCount: 1 }
```

12. DELETE MANY OPERATION

MongoDB Query:

```
db.students.deleteMany({
  marks: { $lt: 60 }
})
```

Explanation: Deletes all students having marks below 60.

```
MongoDB Shell — Students

> MONGOSH
db.students.deleteMany({
  marks: { $lt: 60 }
})
{ acknowledged: true, deletedCount: 8 }
```

13. AND OPERATION --- \$and

MongoDB Query:

```
db.students.find({
  $and: [
    { marks: { $gt: 80 } },
    { age: 19 }
  ]
})
```

Explanation: Finds students satisfying both conditions.

```
MongoDB Shell — Students

> MONGOSH
db.students.find({
  $and: [
    { marks: { $gt: 80 } },
    { age: 19 }
  ]
})
{ _id: ObjectId('6f1a2e9c78f0f98eb52df216'), roll: 23, name: 'Radhika', age: 19, marks: 92, city: 'Ayodhya' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df21a'), roll: 27, name: 'Anjali', age: 19, marks: 93, city: 'Gorakhpur' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df221'), roll: 34, name: 'Arjun', age: 19, marks: 97, city: 'Gonda' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df231'), roll: 50, name: 'Sanya', age: 19, marks: 84, city: 'Gonda' }
```

14. OR OPERATION --- \$or

MongoDB Query:

```
db.students.find({
  $or: [
    { city: "Lucknow" },
    { city: "Kanpur" }
  ]
})
```

Explanation: Finds students from Lucknow or Kanpur.

```
MongoDB Shell — Students

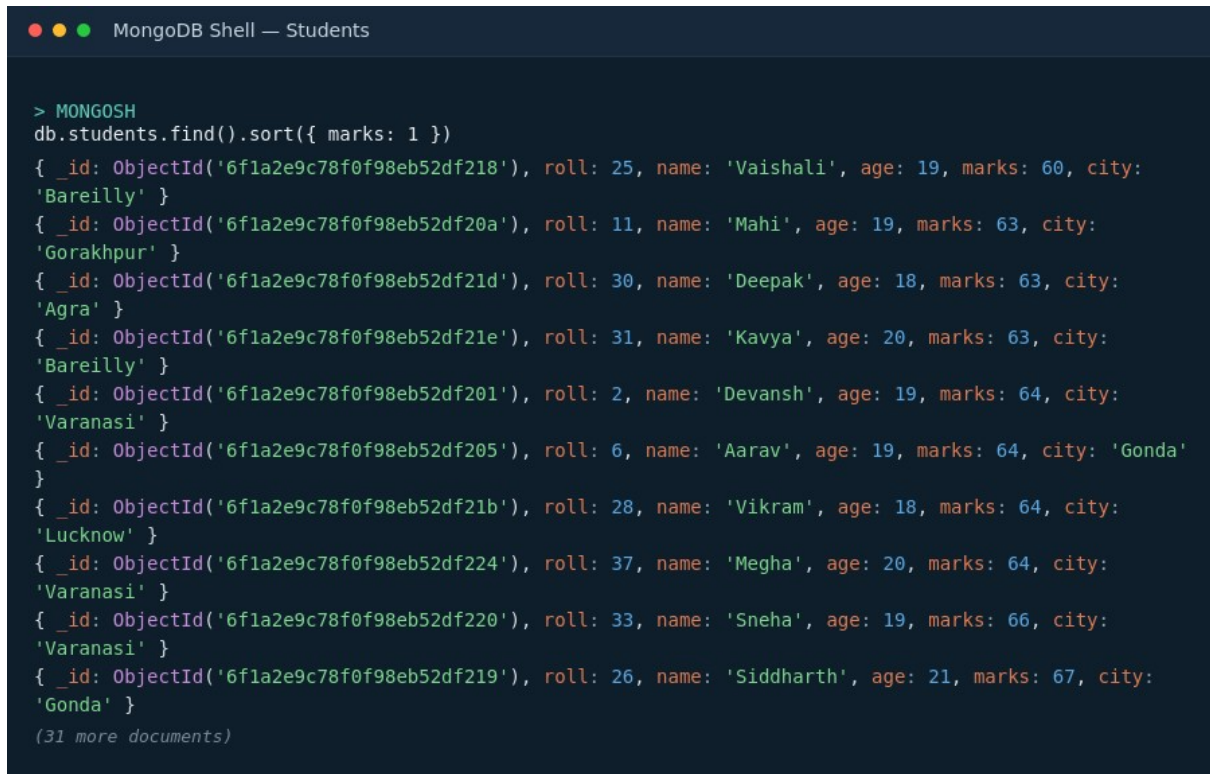
> MONGOSH
db.students.find({
  $or: [
    { city: "Lucknow" },
    { city: "Kanpur" }
  ]
})
{ _id: ObjectId('6f1a2e9c78f0f98eb52df204'), roll: 5, name: 'Kritika', age: 18, marks: 95, city: 'Kanpur' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df20e'), roll: 15, name: 'Diksha', age: 18, marks: 74, city: 'Kanpur' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df21b'), roll: 28, name: 'Vikram', age: 18, marks: 64, city: 'Lucknow' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df227'), roll: 40, name: 'Gaurav', age: 21, marks: 88, city: 'Kanpur' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df22a'), roll: 43, name: 'Rohan', age: 18, marks: 93, city: 'Kanpur' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df22d'), roll: 46, name: 'Charu', age: 19, marks: 79, city: 'Lucknow' }
```

15. SORT OPERATION (Ascending)

MongoDB Query:

```
db.students.find().sort({ marks: 1 })
```

Explanation: Sorts students by marks from lowest to highest. Use 1 for ascending order.

A screenshot of the MongoDB Shell interface. The title bar reads "MongoDB Shell — Students". The prompt is "> MONGOSH". The command entered is "db.students.find().sort({ marks: 1 })". The output shows a list of student documents sorted by marks in ascending order. The first document has marks: 60, the second 63, the third 63, the fourth 63, the fifth 64, the sixth 64, the seventh 64, the eighth 66, and the ninth 67. The output is truncated with "(31 more documents)".

```
> MONGOSH
db.students.find().sort({ marks: 1 })
{ _id: ObjectId('6f1a2e9c78f0f98eb52df218'), roll: 25, name: 'Vaishali', age: 19, marks: 60, city: 'Bareilly' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df20a'), roll: 11, name: 'Mahi', age: 19, marks: 63, city: 'Gorakhpur' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df21d'), roll: 30, name: 'Deepak', age: 18, marks: 63, city: 'Agra' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df21e'), roll: 31, name: 'Kavya', age: 20, marks: 63, city: 'Bareilly' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df201'), roll: 2, name: 'Devansh', age: 19, marks: 64, city: 'Varanasi' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df205'), roll: 6, name: 'Aarav', age: 19, marks: 64, city: 'Gonda' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df21b'), roll: 28, name: 'Vikram', age: 18, marks: 64, city: 'Lucknow' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df224'), roll: 37, name: 'Megha', age: 20, marks: 64, city: 'Varanasi' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df220'), roll: 33, name: 'Sneha', age: 19, marks: 66, city: 'Varanasi' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df219'), roll: 26, name: 'Siddharth', age: 21, marks: 67, city: 'Gonda' }
(31 more documents)
```

15. SORT OPERATION (Descending)

MongoDB Query:

```
db.students.find().sort({ marks: -1 })
```

Explanation: Sorts students by marks from highest to lowest. Use -1 for descending order.

```
MongoDB Shell — Students

> MONGOSH
db.students.find().sort({ marks: -1 })
{ _id: ObjectId('6f1a2e9c78f0f98eb52df22b'), roll: 44, name: 'Aditi', age: 20, marks: 99, city: 'Gorakhpur' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df221'), roll: 34, name: 'Arjun', age: 19, marks: 97, city: 'Gonda' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df222'), roll: 35, name: 'Tanvi', age: 20, marks: 97, city: 'Agra' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df204'), roll: 5, name: 'Kritika', age: 18, marks: 95, city: 'Kanpur' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df202'), roll: 3, name: 'Tanya', age: 18, marks: 93, city: 'Gonda' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df21a'), roll: 27, name: 'Anjali', age: 19, marks: 93, city: 'Gorakhpur' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df22a'), roll: 43, name: 'Rohan', age: 18, marks: 93, city: 'Kanpur' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df216'), roll: 23, name: 'Radhika', age: 19, marks: 92, city: 'Ayodhya' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df21f'), roll: 32, name: 'Nikhil', age: 21, marks: 91, city: 'Bareilly' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df20f'), roll: 16, name: 'Rajat', age: 20, marks: 90, city: 'Agra' }
(31 more documents)
```

16. LIMIT OPERATION

MongoDB Query:

```
db.students.find().limit(5)
```

Explanation: Displays only the first five documents.

```
MongoDB Shell — Students

> MONGOSH
db.students.find().limit(5)
{ _id: ObjectId('6f1a2e9c78f0f98eb52df201'), roll: 2, name: 'Devansh', age: 19, marks: 64, city: 'Varanasi' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df202'), roll: 3, name: 'Tanya', age: 18, marks: 93, city: 'Gonda' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df203'), roll: 4, name: 'Yash', age: 18, marks: 87, city: 'Jaunpur' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df204'), roll: 5, name: 'Kritika', age: 18, marks: 95, city: 'Kanpur' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df205'), roll: 6, name: 'Aarav', age: 19, marks: 64, city: 'Gonda' }
```

17. SEARCH BY CITY

MongoDB Query:

```
db.students.find({ city: "Lucknow" })
```

Explanation: Finds all students whose city is Lucknow.

```
MongoDB Shell — Students

> MONGOSH
db.students.find({ city: "Lucknow" })
{ _id: ObjectId('6f1a2e9c78f0f98eb52df21b'), roll: 28, name: 'Vikram', age: 18, marks: 64, city: 'Lucknow' }
{ _id: ObjectId('6f1a2e9c78f0f98eb52df22d'), roll: 46, name: 'Charu', age: 19, marks: 79, city: 'Lucknow' }
```

18. COUNT DOCUMENTS

MongoDB Query:

db.students.countDocuments()

Explanation: Counts documents currently present in the collection (performed after the delete operations).

```
MongoDB Shell — Students

> MONGOSH
db.students.countDocuments()
41
```

MONGODB OPERATORS USED

Operator	Meaning	Example
\$eq	Equal to	marks: { \$eq: 88 }
\$gt	Greater than	marks: { \$gt: 85 }
\$lt	Less than	marks: { \$lt: 60 }
\$gte	Greater than or equal to	marks: { \$gte: 90 }
\$lte	Less than or equal to	marks: { \$lte: 70 }
\$and	All conditions true	\$and: [...]
\$or	At least one condition true	\$or: [...]

CONCLUSION

This project provided practical knowledge of NoSQL databases using MongoDB. A Student Management System containing student information was created and different MongoDB operations were performed on the data.

Through this project, I learned how to insert, retrieve, update, and delete documents. I also learned how to sort data in ascending and descending order, how comparison operators such as \$eq, \$gt, \$lt, \$gte, and \$lte work, along with logical operators such as \$and and \$or.

Overall, this project helped me understand how MongoDB can be used to manage and query student data efficiently.